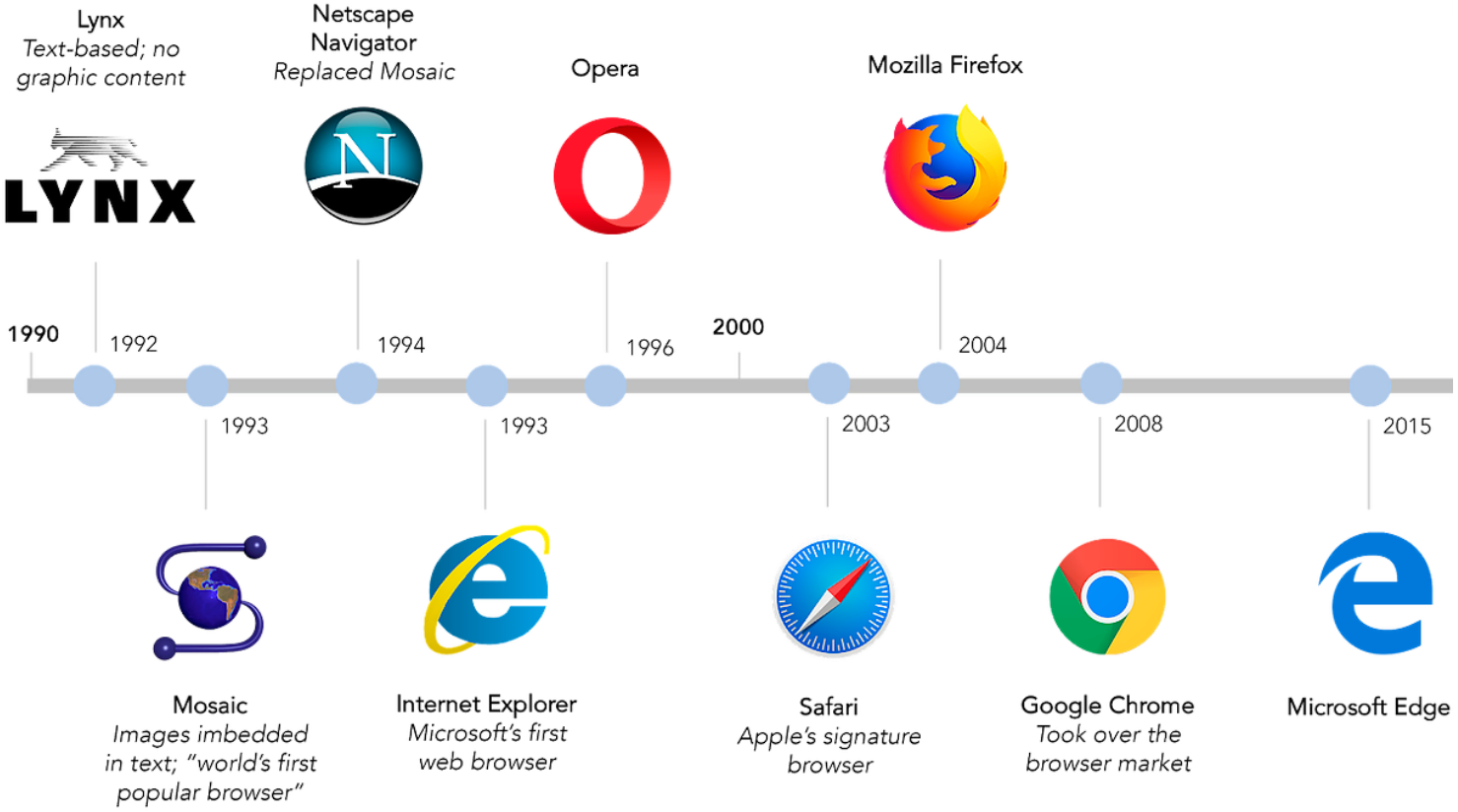
Webtechnologien

Basic Frontends - CSS



Stephan Schiffner

Geschichte der Web Browser



https://medium.com/fulldive/a-brief-history-of-browsers-9e8f453dbf45

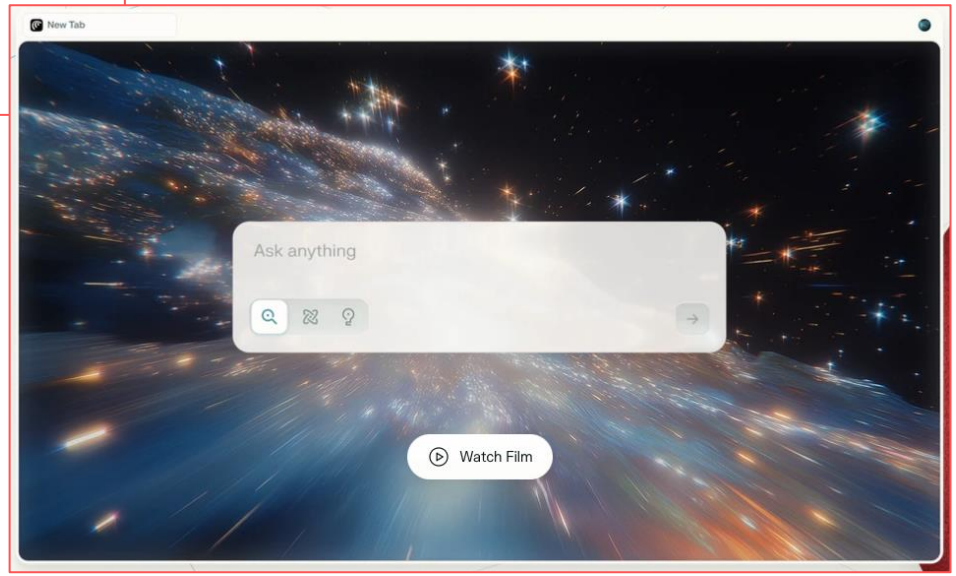
KI-basierte Web Browser: Atlas + Comet

October 21, 2025 Product

Introducing ChatGPT Atlas

The browser with ChatGPT built in.

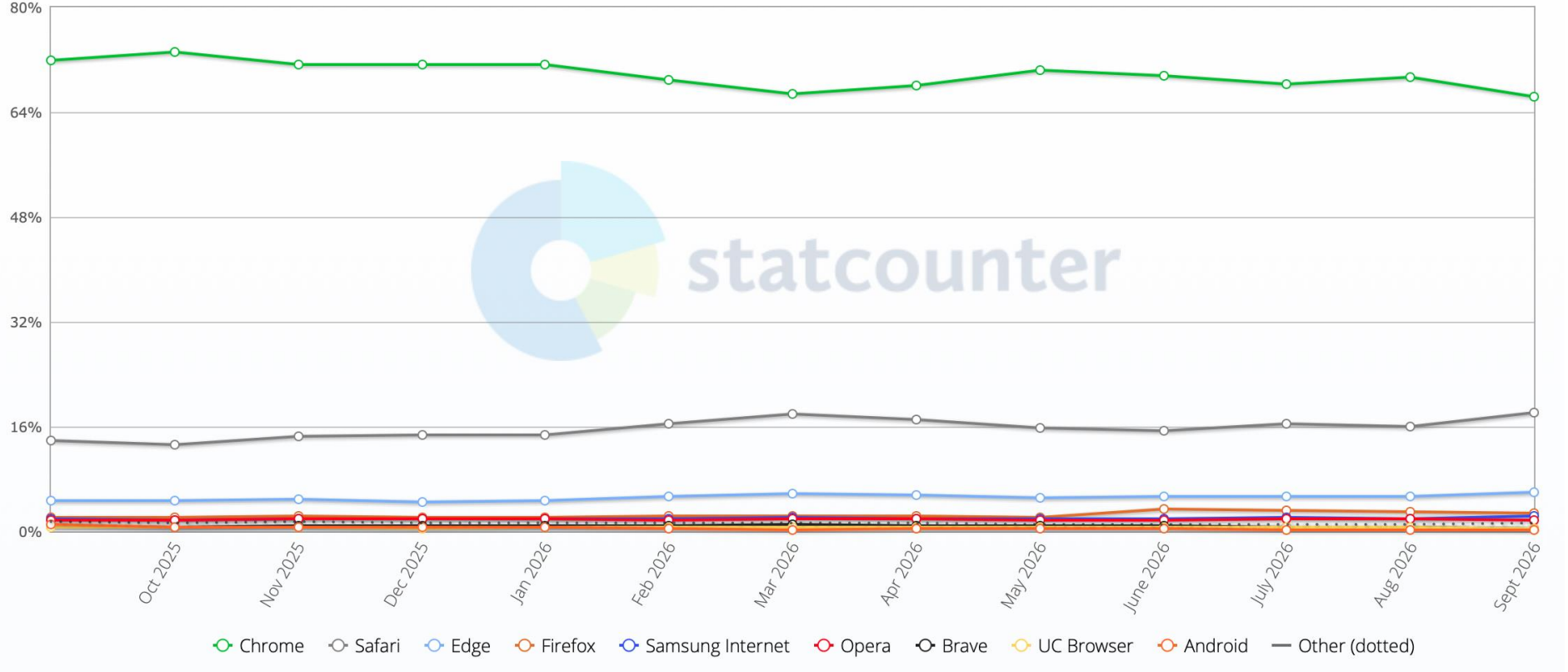
Download for macOS ↗



Browser Market Share Worldwide

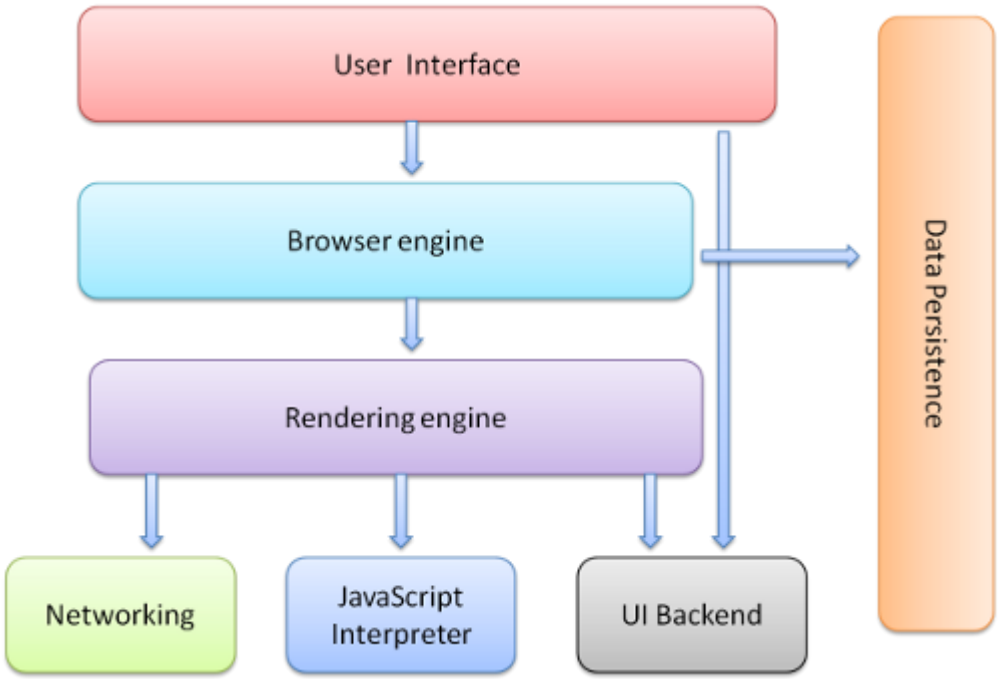
Sept 2025 - Sept 2026

Edit Chart Data



https://gs.statcounter.com/browser-market-share

Browser Aufbau



https://web.dev/howbrowserswork/

Browser Komponenten

User Interface: alles außer der Anzeige des Web-Contents

Browser Engine: Verbindung zwischen User Interface und Rendering Engine

Rendering Engine: Darstellung Web-Content (HTML, CSS, Video, u.a.)

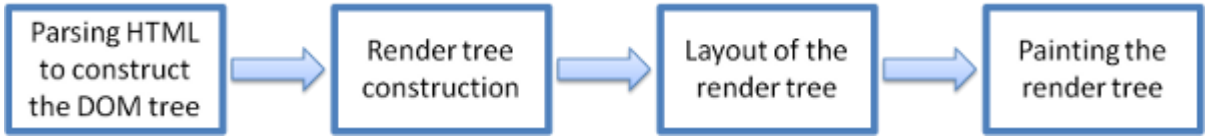
Networking: HTTP requests

UI backend: Generisches Interface zum Betriebssystem und dem Plattform-spezifischen User Interface

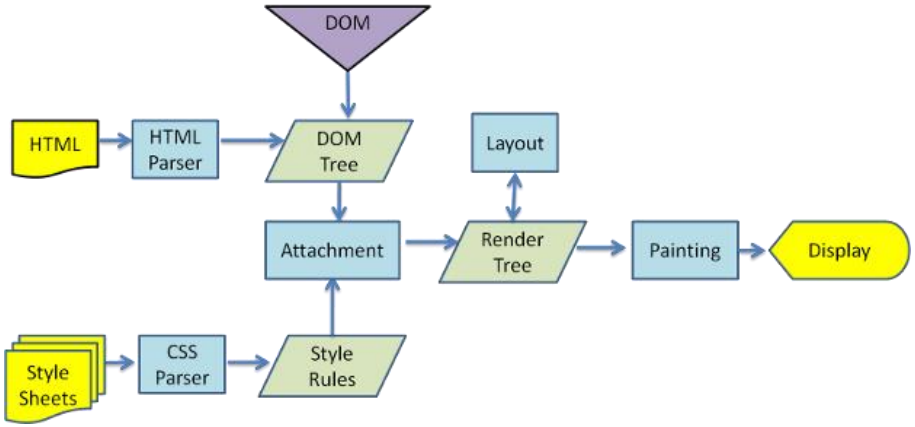
JavaScript interpreter. Parst und führt JavaScript Code aus

Data storage. Der ‘persistence layer’, Datenspeicherung von Cookies und anderen Daten. Browser verwenden neben eigenem Datenspeicher auch das lokale Dateisystem.

Rendering Engine



Rendering Engine Basic Flow (https://web.dev/howbrowserswork/)

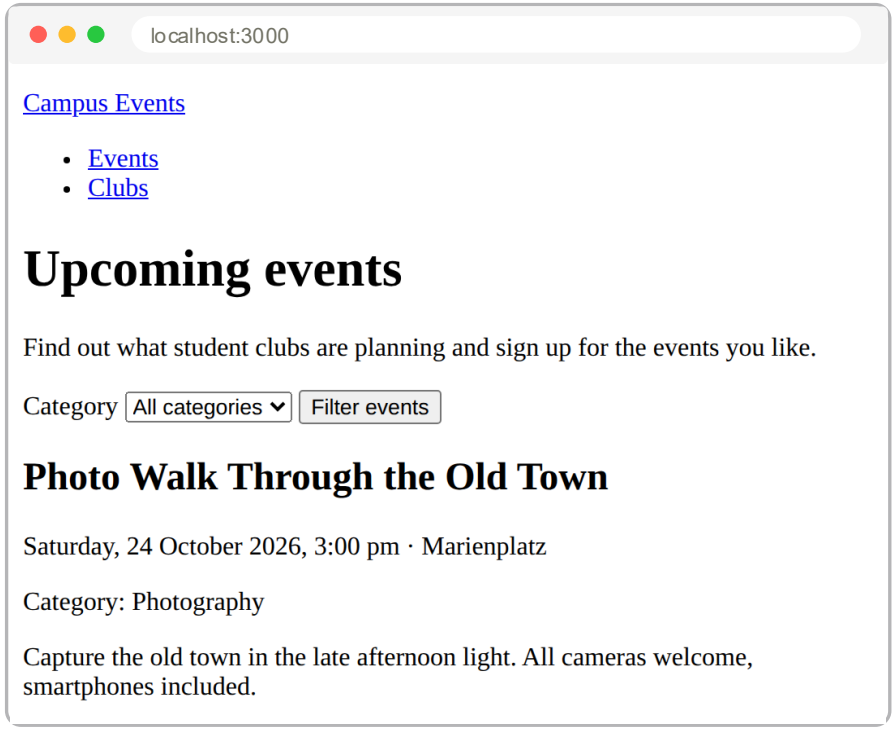


Beispiel WebKit Main Flow (WebKit = Rendering Engine von Safari)

Begriffe:

- **HTML**: Die Hypertext Markup Language - textbasierte Auszeichnungssprache zur Strukturierung elektronischer Dokumente wie Texte mit Hyperlinks, Bildern und anderen Inhalten.
- **CSS**: Cascading Style Sheets - Darstellungsvorgaben für HTML oder XML Elemente
- **DOM**: Document Object Model - stellt HTML oder XML Dokumente als Baumstruktur dar

Rückblick: Stand nach Vorlesung 2



Offene Fäden

Sieht schlicht aus — heute
CSS

Der Filter tut nichts — Block 2
JavaScript

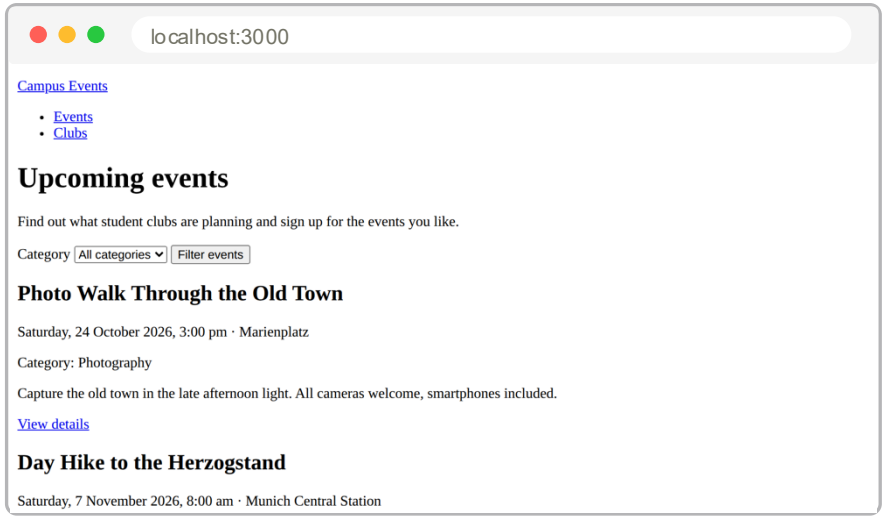
Header in jeder Datei kopiert — Block 3
Komponenten

Anmeldung endet mit 501 — Block 4
Backend

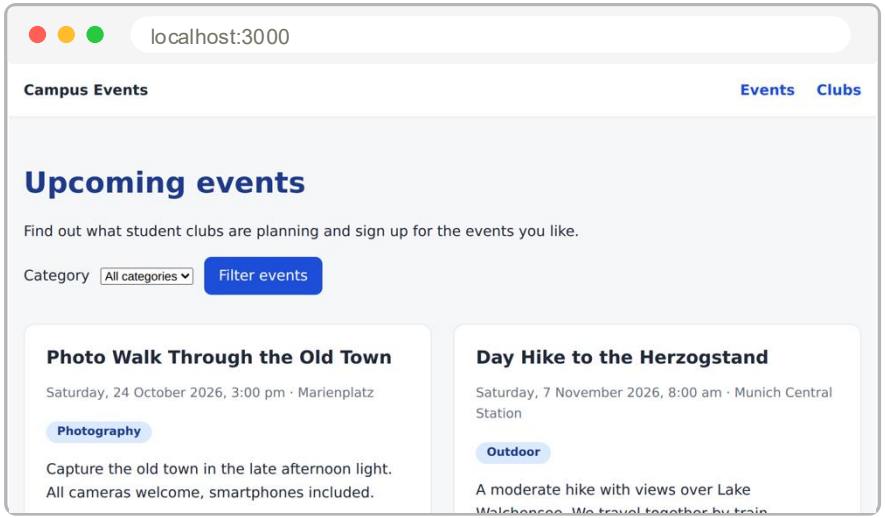
Was ist CSS?

Cascading Style Sheets – Regeln, die beschreiben, wie Elemente aussehen.

`step-5-forms`



`step-10-responsive`



Gleicher Inhalt, gleiche Struktur – nur ein Stylesheet und ein paar Klassen sind dazugekommen.

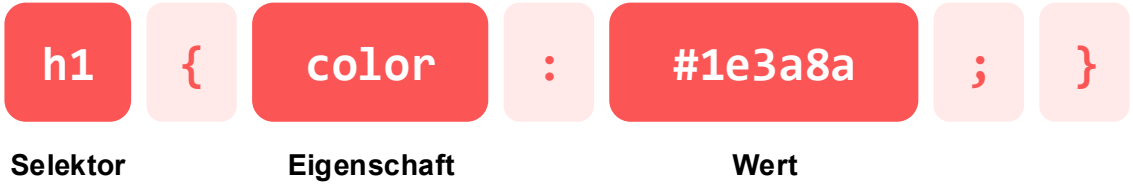
CSS einbinden

Externe Datei	`<link rel="stylesheet" href="css/style.css">`	Eine Datei für alle Seiten – der Standard.
Im head	`<style> h1 { color: navy; } </style>`	Gilt nur für diese eine Seite.
Am Element	`<h1 style="color: navy">…</h1>`	Gilt nur für dieses Element – schwer zu pflegen.



Ein weiterer Request: Im Network-Tab erscheint style.css mit **`Content-Type: text/css`**

Aufbau einer CSS-Regel



Deklaration
Eigenschaft und Wert: color: #1e3a8a;

Regel
Selektor plus alle Deklarationen in geschweiften Klammern

Kommentar
/* Erklärung für Menschen */ – der Browser ignoriert ihn

Eine Regel in Aktion

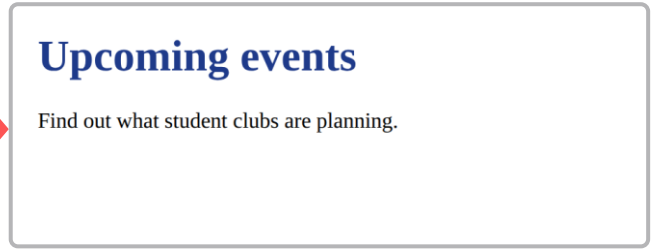
HTML `index.html`

```html
<h1>Upcoming events</h1>
<p>Find out what student
   clubs are planning.</p>
```

CSS `css/style.css`

```css
h1 {
  color: #1e3a8a;
}
```

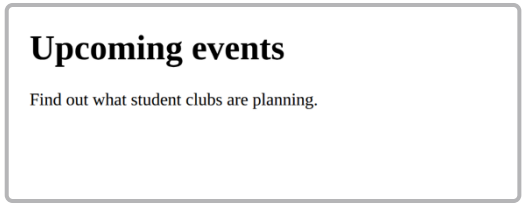
Ergebnis im Browser mit CSS



1 Selektor
h1 wählt alle h1-Elemente aus – der Absatz ist nicht gemeint.

2 Deklaration
color: #1e3a8a; setzt die Textfarbe der ausgewählten Elemente.

zum Vergleich: ohne CSS



Die Überschrift wird blau, der Absatz bleibt unverändert – der Selektor entscheidet, was betroffen ist.

Selektoren

Selektor	Trifft	Beispiel
Element	alle Elemente dieses Typs	`h1`
Klasse	alle Elemente mit dieser Klasse	`.event-card`
ID	genau ein Element – nicht fürs Styling	`#shutter-society`
Nachfahre	Elemente innerhalb eines anderen	`.site-nav a`
Pseudo-Klasse	einen Zustand	`a:hover` `:focus`
Gruppe	mehrere Selektoren auf einmal	`h1, h2`

Eine Klasse in Aktion

HTML `index.html`

```html
<h1>Photo Walk</h1>
<p class="event-meta">
  Saturday, 24 October 2026</p>
<p>Capture the old town …</p>
```

CSS `css/style.css`

```css
.event-meta {
  color: #6b7280;
  font-size: 0.9rem;
}
```

Ergebnis im Browser `.event-meta`



zum Vergleich: p statt `.event-meta`



1. **Im HTML: ohne Punkt**
   class="event-meta" hängt die Klasse an genau dieses Element.

2. **Im CSS: mit Punkt**
   .event-meta wählt alle Elemente mit dieser Klasse aus.

Nur der Absatz mit der Klasse ändert sich – der andere p bleibt, wie er ist.

Eine Pseudo-Klasse in Aktion

HTML `index.html`

```html
<a class="button" href="…">
  View details
</a>
```

CSS `css/style.css`

```css
.button {
  background-color: #1d4ed8;
}
.button:hover {
  background-color: #1e3a8a;
}
.button:focus-visible {
  outline: 3px solid #fc5555;
}
```

Ergebnis im Browser



normal

`:hover`
Maus darüber

`:focus-visible`
per Tab ausgewählt

1 Zustand statt Element
Pseudo-Klassen gelten, solange ein Element in einem bestimmten Zustand ist.

2 Mit Doppelpunkt
.button:hover – direkt hinter dem Selektor, ohne Leerzeichen.

Weitere:
`:visited` `:first-child` `:nth-child(2)`

Der Browser schaltet die Regel ein, sobald der Zustand eintritt – ganz ohne JavaScript.

Kaskade, Spezifität und Vererbung

Spezifität

Je genauer der Selektor, desto stärker.

```
style-Attribut
#id
  .klasse  :hover
  element
```

Reihenfolge

Bei gleicher Spezifität gewinnt die spätere Regel.

```
a { color: blue; }
a { color: navy; }  ← gewinnt
```

Vererbung

Manche Eigenschaften gehen auf Kinder über.

```
erbt: color, font-family
erbt nicht: margin, border
```

Von stark nach schwach: `style-Attribut` › `ID` › `Klasse/Pseudo-Klasse` › `Element`

Spezifität: die Stufen im Beispiel

```
<a id="nav-events" class="nav-link" href="#" style="color: #dc2626">Events</a>
```

Stufe von schwach nach stark	Regel	Spezifität	Ergebnis danach
1 **Element**	`a { color: #6b7280; }`	0-0-1	Events
2 **Klasse / Pseudo-Klasse**	`.nav-link { color: #1d4ed8; }`	0-1-0	Events
3 **ID**	`#nav-events { color: #15803d; }`	1-0-0	Events
4 **style-Attribut**	`style="color: #dc2626"`	schlägt alle	Events

Jede Stufe schlägt alle vorherigen – auch zehn Klassen schlagen keine ID.

Vererbung in Aktion

HTML `index.html`

```html
<article class="event-card">
  <h2>Photo Walk</h2>
  <p>Capture the old …</p>
  <p><a href="#">View
    details</a></p>
</article>
```

CSS `css/style.css`

```css
.event-card {
  color: #1e3a8a;
  font-family: system-ui;
  border: 3px solid #fc5555;
}
```

Ergebnis im Browser



Wird vererbt
color, font-family, font-size, line-height – Textgestaltung geht auf die Kinder über.

Wird nicht vererbt
border, padding, margin, background – gilt nur für das Element selbst.

Der Link bleibt blau: Der Browser hat eine eigene Regel für a. Jede Regel direkt am Element schlägt einen geerbten Wert.

Eine Regel an der Karte – Überschrift und Absatz erben Farbe und Schrift, aber keinen Rahmen.

Campus Events Status

Campus Events

localhost:3000

Campus Events

- Events
- Clubs

Upcoming events

Find out what student clubs are planning and sign up for the events you like.

Category All categories Filter events

Photo Walk Through the Old Town

Saturday, 24 October 2026, 3:00 pm · Marienplatz

Category: Photography

Capture the old town in the late afternoon light. All cameras welcome, smartphones inclu

View details

Campus Events

- Events
- Clubs

Photo Walk Through the Old Town



Sign up

Name

Email

Study program Computer Science

I agree that my name and email are shared with the organizer.

Sign up

Version 6: Typografie und Farben

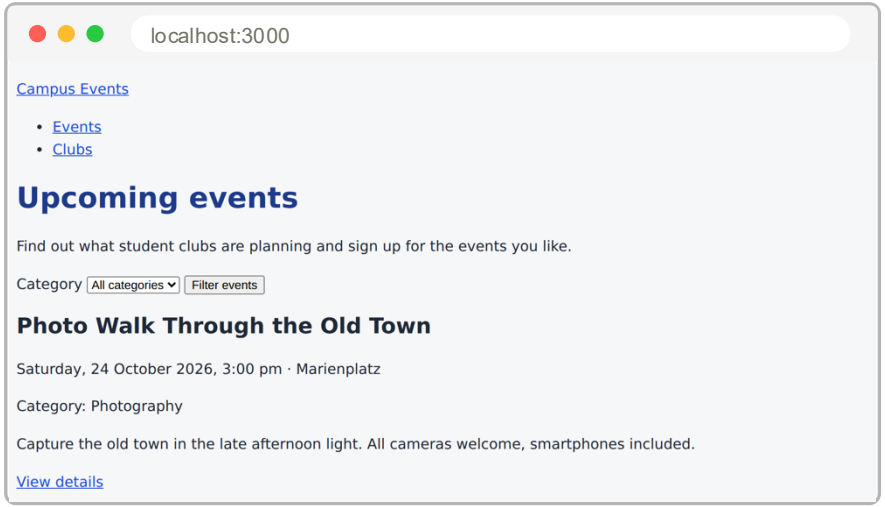
`step-6-stylesheet`

css/style.css (Auszug)

```css
:root {
  --color-primary: #1d4ed8;
  --color-text: #1f2937;
  --font-body: system-ui, sans-serif;
}

body {
  font-family: var(--font-body);
  line-height: 1.6;
  color: var(--color-text);
}

a {
  color: var(--color-primary);
}
```



`--color-primary` einmal definiert, überall mit `var()` verwendet.

Warum greift meine Regel nicht?

HM

css/style.css

```css
a {
  color: var(--color-primary);
}

.site-name a {
  color: var(--color-text);
}
```

DevTools – Styles

```
.site-name a {
  color: var(--color-text);
}

a {
  color: var(--color-primary); überschrieben
}
```

Klasse + Element
schlägt Element allein – unabhängig von der Reihenfolge

Durchgestrichen
heißt: Regel trifft, wird aber überschrieben

Fehlt ganz?
dann trifft der Selektor das Element nicht – Tippfehler prüfen

Welche Farbe?

HTML

```html
<nav class="site-nav">
  <a class="active" href="#">Events</a>
</nav>
```

CSS – in genau dieser Reihenfolge

```css
.site-nav a {
  color: black;
}

a {
  color: blue;
}

.active {
  color: red;
}
```

Welche Farbe hat „Events“?

A **blau**

B **schwarz**

C **rot**

Auflösung: schwarz

Selektor	IDs	Klassen	Elemente	Ergebnis
`.site-nav a`	0	1	1	**gewinnt**
`.active`	0	1	0	**verliert**
`a`	0	0	1	**verliert**

Drei Zahlen
IDs – Klassen – Elemente, verglichen von links nach rechts.

0-1-1 schlägt 0-1-0
Die zweite Stelle ist gleich, die dritte entscheidet.

Reihenfolge
zählt nur bei Gleichstand – .active steht zuletzt und verliert trotzdem.

Eine ID in Aktion

HTML `clubs.html`

```html
<ul>
  <li class="club-card"
      id="shutter-society">
    Shutter Society</li>
  <li class="club-card">
    Alpine Wanderers</li>
  …
```

CSS `css/style.css`

```css
.club-card {
  border: 2px solid #e5e7eb;
}

#shutter-society {
  border-color: #fc5555;
  background-color: #ffe9e9;
}
```

Ergebnis im Browser `#shutter-society`



Wofür IDs gut sind: Sprungmarken wie `clubs.html#shutter-society` und `label for` – nicht fürs Styling.

1 **Im HTML: einmal pro Seite**
id="shutter-society" – jede ID darf auf einer Seite nur einmal vorkommen.

2 **Im CSS: mit Raute**
#shutter-society – Spezifität 1-0-0, schlägt jede Klasse.

Funktioniert – aber 1-0-0 lässt sich kaum noch überschreiben. Deshalb: Klassen fürs Styling.

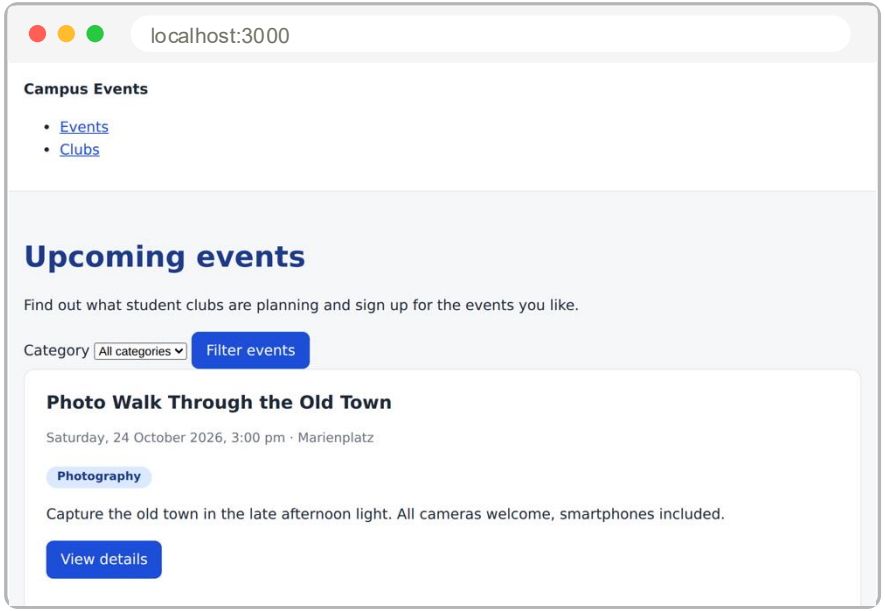
Version 7: Klassen und Karten

`step-7-classes-box-model`

css/style.css (Auszug)

```css
.event-card {
  padding: 1.5rem;
  margin-bottom: 1.5rem;
  background-color: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius);
}

.category-badge {
  display: inline-block;
  padding: 0.125rem 0.75rem;
  border-radius: 999px;
}
```



Block, inline und inline-block

block neue Zeile, volle Breite – h1, p, div, nav

h2 – Überschrift

p – Absatz

nav – Navigation

inline im Textfluss, so breit wie der Inhalt – a, span, strong

Ein Absatz mit einem Link, einem **fetten Wort** und einem span – alles im Textfluss.

padding bei inline und inline-block

Erste Zeile mit etwas Text davor.
Kategorie: **Photography** im Text
Dritte Zeile mit etwas Text danach.

inline: überlappt die Nachbarzeilen

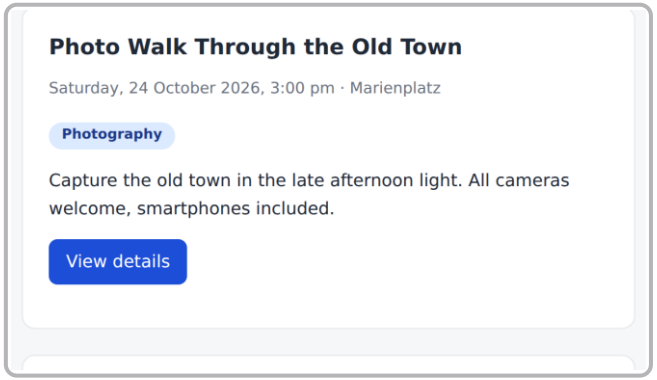
Erste Zeile mit etwas Text davor.
Kategorie: **Photography** im Text
Dritte Zeile mit etwas Text danach.

inline-block: die Zeile macht Platz

Das Box-Modell an der Event-Karte

Jedes Element ist eine Box – das Box-Modell bestimmt Abstände, Rahmen und Größe.

So sieht man sie



So ist sie aufgebaut



`content`
der Inhalt

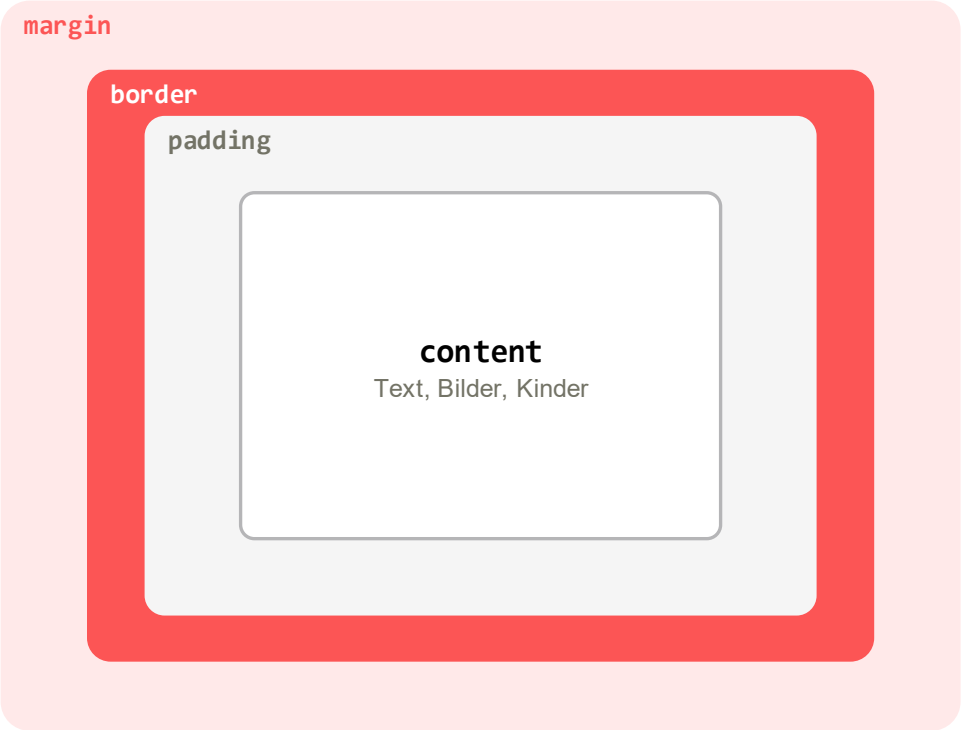
`padding`
Text klebt nicht am Rand
`1.5rem`

`border`
macht die Karte sichtbar
`1px`

`margin`
Abstand zur nächsten Karte
`1.5rem unten`

Jede Abstandsfrage ist eine Frage an eine dieser Schichten – die DevTools zeigen, an welche.

Das Box-Modell



box-sizing

`content-box` (Standard): width gilt nur für den Inhalt – padding und border kommen dazu.

`border-box`: width gilt inklusive padding und border.

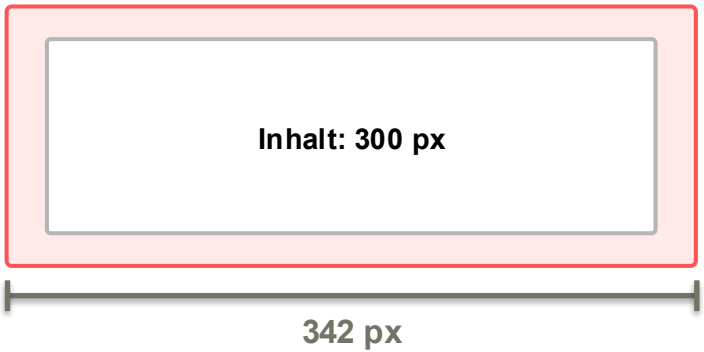
am Anfang jedes Stylesheets

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

Das Box-Modell in Zahlen

Gleiche Regel: `width: 300px; padding: 20px; border: 1px solid;`

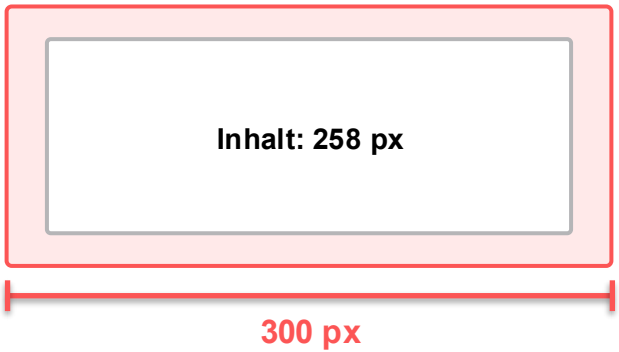
`content-box (Standard)`




```
.box {
  width: 300px;
  padding: 20px;
  border: 1px solid;
}
```

`300 + 2 × 20 + 2 × 1 = 342 px`

`border-box`

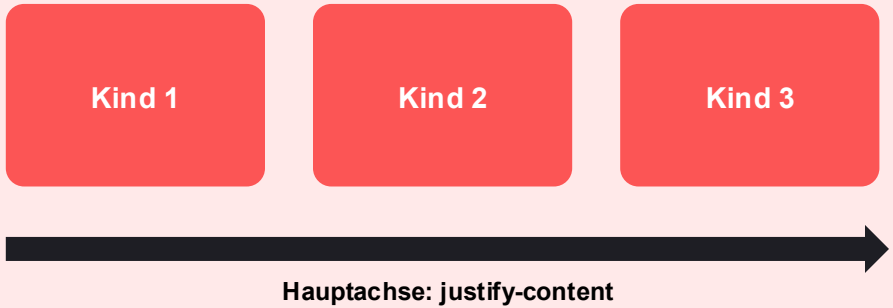



```
.box {
  box-sizing: border-box;
  width: 300px; padding: 20px;
  border: 1px solid;
}
```

`Box genau 300 px – Inhalt 300 – 40 – 2 = 258 px`

Flexbox: Layout in einer Richtung

```
display: flex;
```



`display: flex`	macht ein Element zum Flex-Container
`flex-direction`	row (Standard) oder column
`justify-content`	Verteilung entlang der Hauptachse
`align-items`	Ausrichtung entlang der Querachse
`gap`	Abstand zwischen den Kindern

Faustregel: Flexbox für eine Reihe von Dingen – Navigation, Header, Buttons.

Schritt für Schritt: der Header

Flex-Kind des Headers Listeneintrag nur zur Veranschaulichung

Version 7

Block-Elemente stapeln sich, jedes so breit wie möglich.



1

```
.site-header {
  display: flex;
```

Kinder nebeneinander – Standard ist stretch: gleich hoch.



2

```
  justify-content:
    space-between;
```

Freier Platz wandert zwischen die Kinder.



3

```
  align-items: center;
```

Mittig auf der Querachse statt gestreckt.



Schritt für Schritt: die Navigation

Flex-Kind des Header

Listeneintrag

nur zur Veranschaulichung

Stand nach Schritt 3

Die Liste steht noch untereinander.



4

```
.site-nav ul {
  display: flex;
```

Einträge nebeneinander – noch dicht an dicht.



5

```
  gap: 1.5rem;
```

Abstand zwischen den Einträgen, ohne margin.



6

```
  list-style: none;
  margin: 0;  padding: 0;
```

Feinschliff: keine Punkte, keine Standardabstände.



Flexbox wirkt nur auf direkte Kinder – deshalb braucht die Liste ihr eigenes `display: flex`

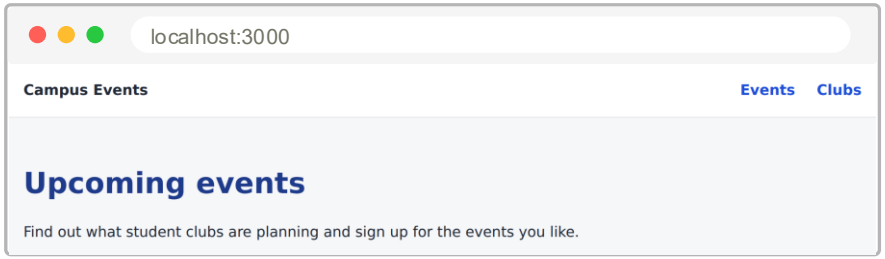
Version 8: Header und Navigation

`step-8-flexbox`

css/style.css (Auszug)

```css
.site-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.site-nav ul {
  display: flex;
  gap: 1.5rem;
  list-style: none;
}
```

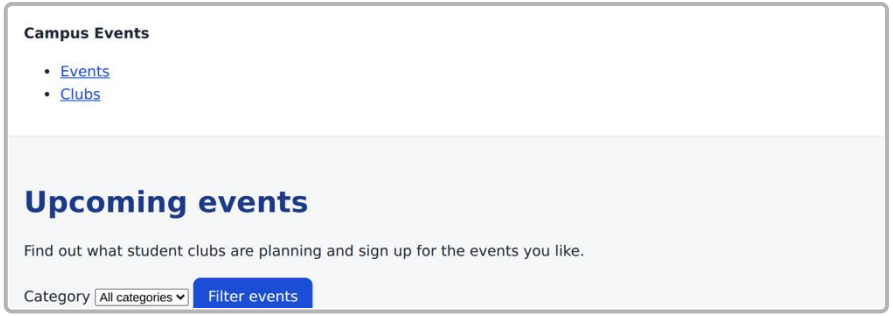


`space-between` schiebt Seitenname und Navigation an die Ränder.

`gap` setzt den Abstand zwischen den Links – ohne margin.

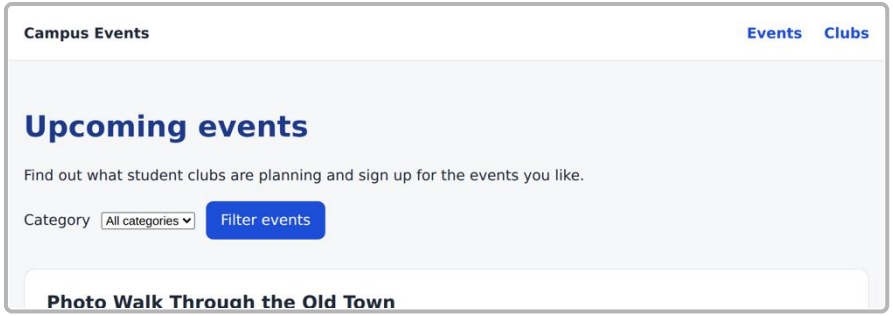
Flexbox: Vorher und nachher

`step-7-classes-box-model`



Block-Elemente stapeln sich: Seitenname und Navigation stehen untereinander.

`step-8-flexbox`

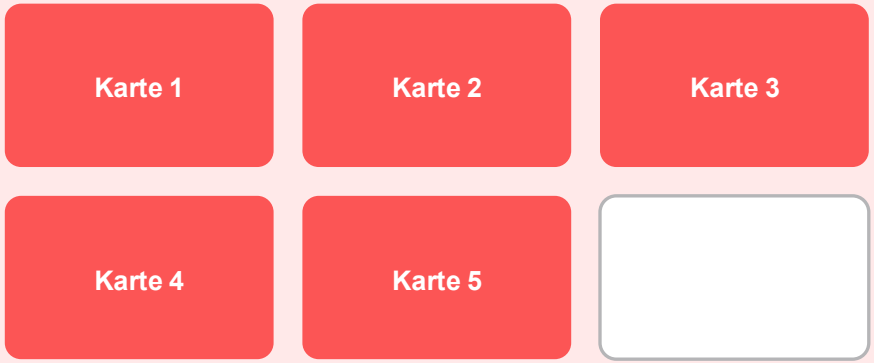


Flexbox: Seitenname links, Navigation rechts – in einer Zeile.

Gleiches HTML – der Unterschied steckt in zwei Containern mit `display: flex`

Grid: Layout in zwei Richtungen

```
display: grid;  grid-template-columns: repeat(3, 1fr);
```



`display: grid`	macht ein Element zum Grid-Container
`grid-template-columns`	legt die Spalten fest
`1fr`	ein Anteil am freien Platz
`repeat(3, 1fr)`	drei gleich breite Spalten
`gap`	Abstand zwischen den Zellen

Flexbox für eine Reihe, Grid für ein Raster – und beide lassen sich kombinieren.

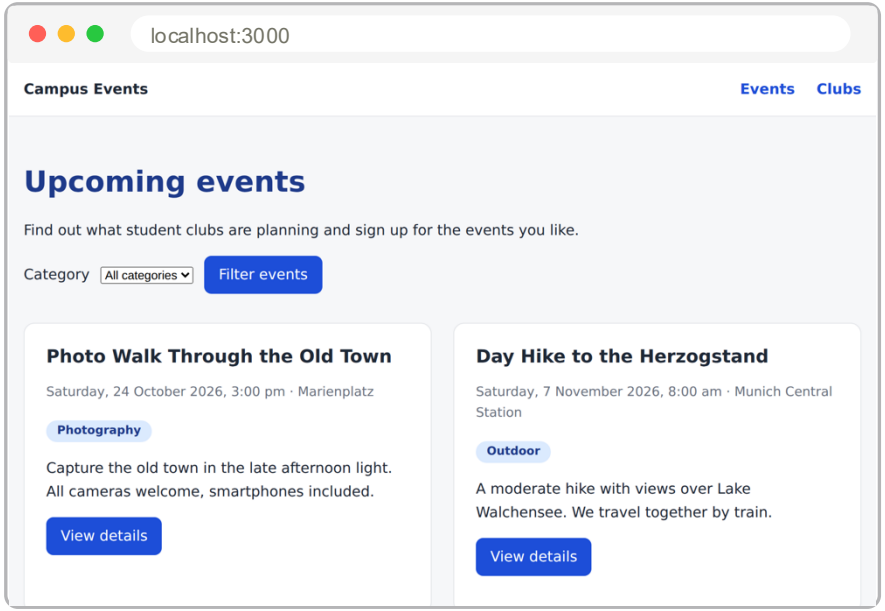
Version 9: Karten im Raster

`step-9-grid`

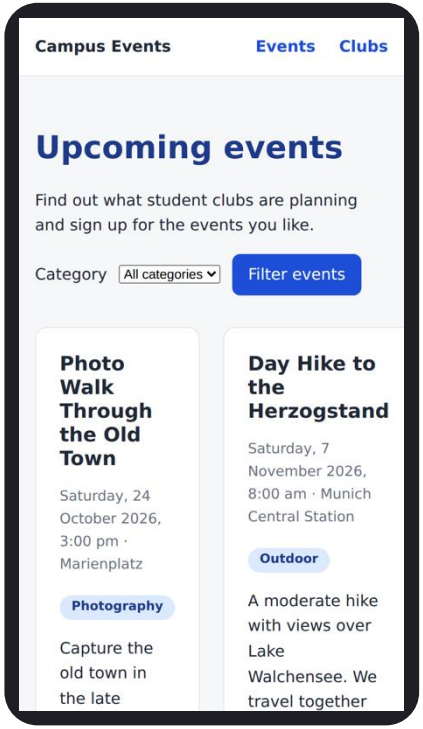
css/style.css (Auszug)

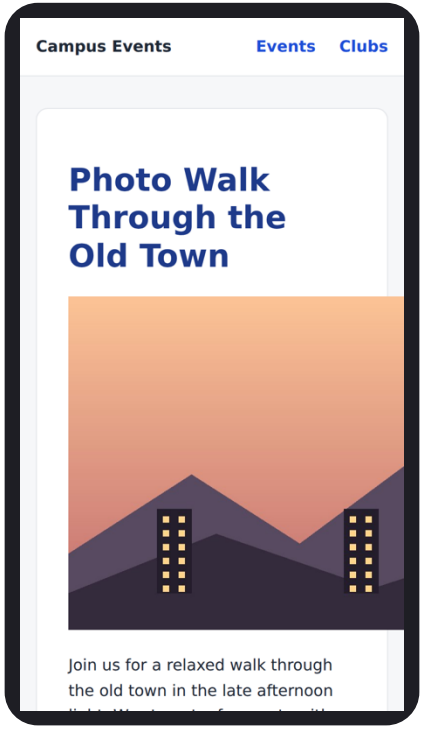
```css
.event-list {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 1.5rem;
}

.event-facts {
  display: grid;
  grid-template-columns: max-content 1fr;
}
```



Version 9 auf dem Smartphone





1. **Zu schmale Spalten**
   Zwei Spalten auf 390 Pixeln – die Texte brechen Wort für Wort um.

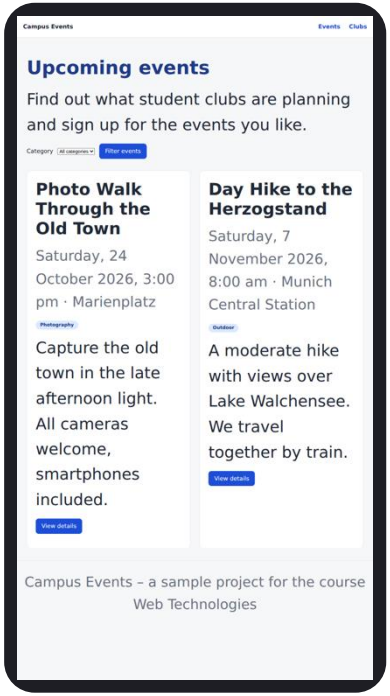
2. **Header ohne Reserve**
   Seitenname und Navigation teilen sich eine Zeile – jeder weitere Link wird eng.

3. **Bild zu breit**
   width="600" ist breiter als der Bildschirm.

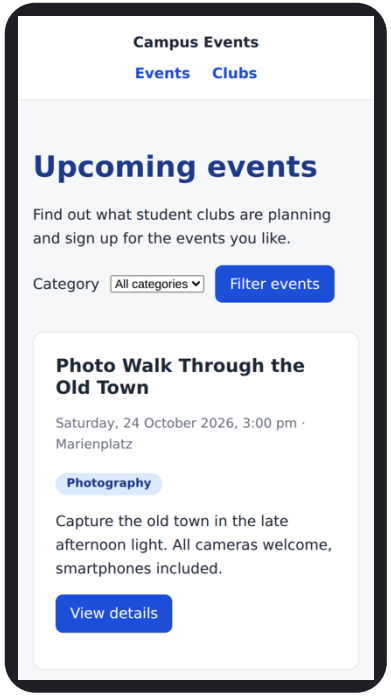
Der viewport-Meta-Tag

HM

ohne viewport



mit viewport




```
<meta name="viewport"
  content="width=device-width, initial-scale=1">
```

Ohne: Das Smartphone rechnet mit rund 980 Pixeln Breite und verkleinert die Seite. Auch die Media Query glaubt, sie sei auf einem Desktop.

Mit: Die Seite ist so breit wie das Gerät – Media Queries und Schriftgrößen funktionieren wie gedacht.

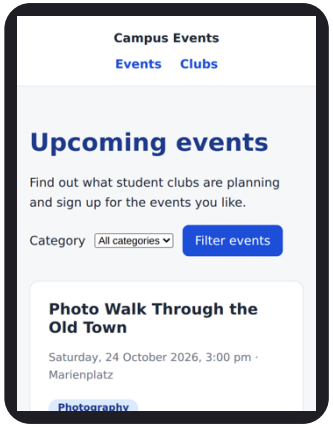
Seit Vorlesung 2 in jeder Seite – jetzt wissen wir, wofür.

Media Queries: ab welcher Breite?

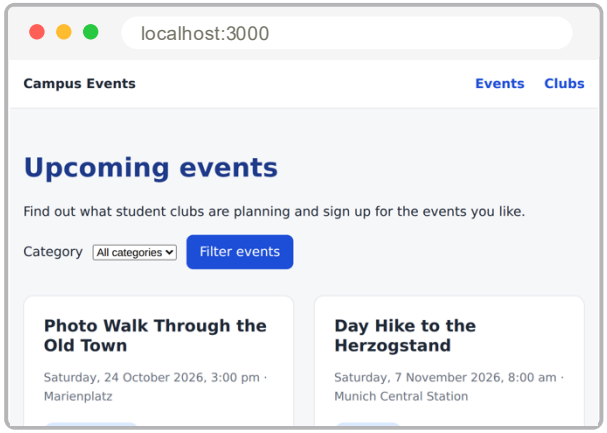
40rem = 640 px

Standardregel: eine Spalte | Media Query greift: zwei Spalten

0 px — 390 px — 720 px — 1280 px



390 px



720 px

immer aktiv – ab 40rem überschrieben

```css
.event-list {
  grid-template-columns: 1fr;
}

@media (min-width: 40rem) {
  .event-list {
    grid-template-columns:
      repeat(2, 1fr);
  }
}
```

Mobile first oder Desktop first?

Desktop first – `max-width`

```
.event-list {
  grid-template-columns: repeat(2, 1fr);
}

@media (max-width: 40rem) {
  .event-list {
    grid-template-columns: 1fr;
  }
}
```

Mobile first – `min-width`

```
.event-list {
  grid-template-columns: 1fr;
}

@media (min-width: 40rem) {
  .event-list {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

Einfache Basis

Eine Spalte braucht kaum CSS – der Standard ist schon fast fertig.

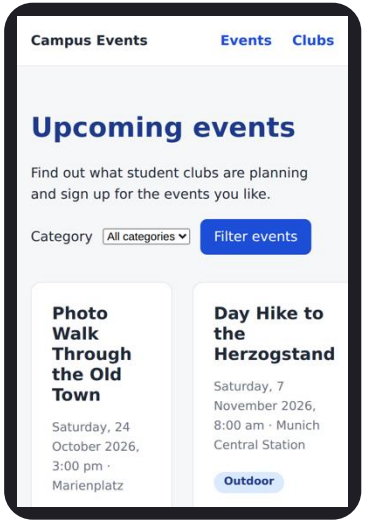
Ergänzen statt zurücknehmen

Für große Bildschirme kommt Layout dazu, statt für kleine überschrieben zu werden.

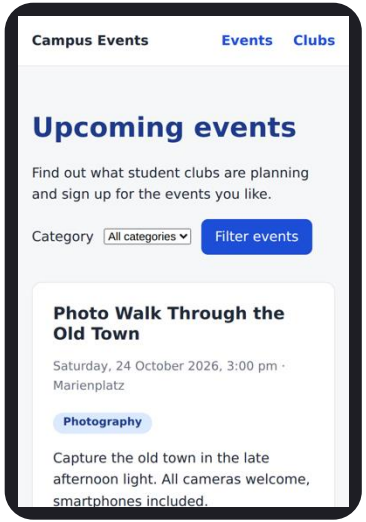
Im Kurs

arbeiten wir durchgehend mobile first.

Schritt für Schritt: Version 9 → 10

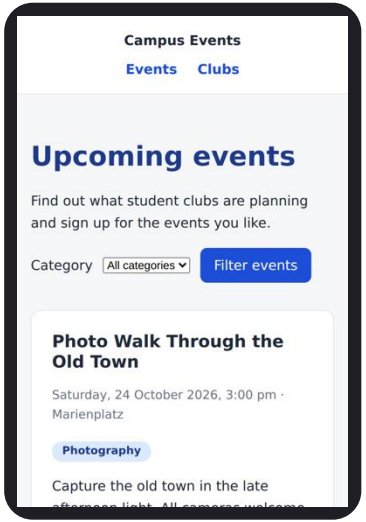


Version 9



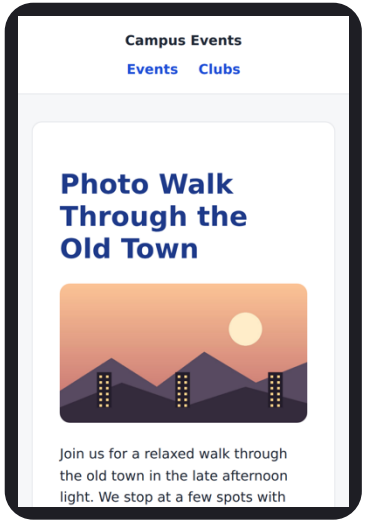
1 **1 Spalte als Standard**

```
.event-list: 1fr
2 Spalten ab 40rem
```



2 **Header gestapelt**

```
flex-direction: column
row ab 40rem
```



3 **Bilder (Detailseite)**

```
img { max-width: 100%;
      height: auto; }
```

Version 10: Mobile first

`step-10-responsive`

Event-Liste – Grid

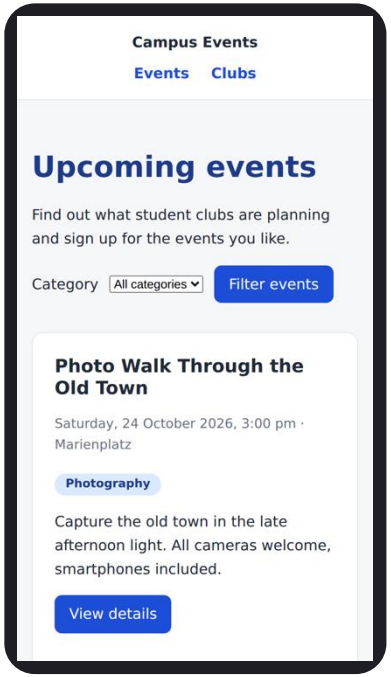
```
.event-list { grid-template-columns: 1fr; }
@media (min-width: 40rem) {
  .event-list { grid-template-columns: repeat(2, 1fr); }
}
```

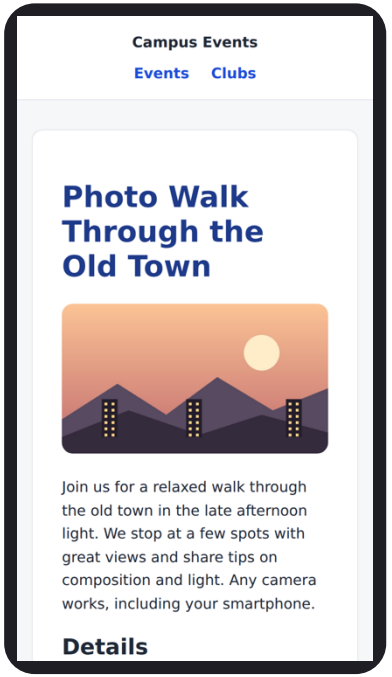
Header – Flexbox

```
.site-header { flex-direction: column; }
@media (min-width: 40rem) {
  .site-header { flex-direction: row; }
}
```

Bilder

```
img { max-width: 100%; height: auto; }
```





Struktur steht, Gestaltung wächst

Version	HTML	CSS
6 Stylesheet	link-Tag	neu
7 Klassen, Box-Modell	Klassen und Container	geändert
8 Flexbox	–	geändert
9 Grid	–	geändert
10 Responsive	–	geändert

Terminal

```
$ git diff --stat \
    step-7-classes-box-model \
    step-10-responsive

 public/css/style.css | 76 ++++++---
 1 file changed, 73 insertions(+),
 3 deletions(-)

# Flexbox, Grid, Responsive:
# nur eine Datei geändert
```

Einmal saubere Struktur – danach entsteht das ganze Layout nur im CSS.

Im Repository: CSS

1. **Stylesheet einbinden**
   `step-6-stylesheet`
2. **Klassen und Box-Modell**
   `step-7-classes-box-model`
3. **Flexbox**
   `step-8-flexbox`
4. **Grid**
   `step-9-grid`
5. **Responsive**
   `step-10-responsive`

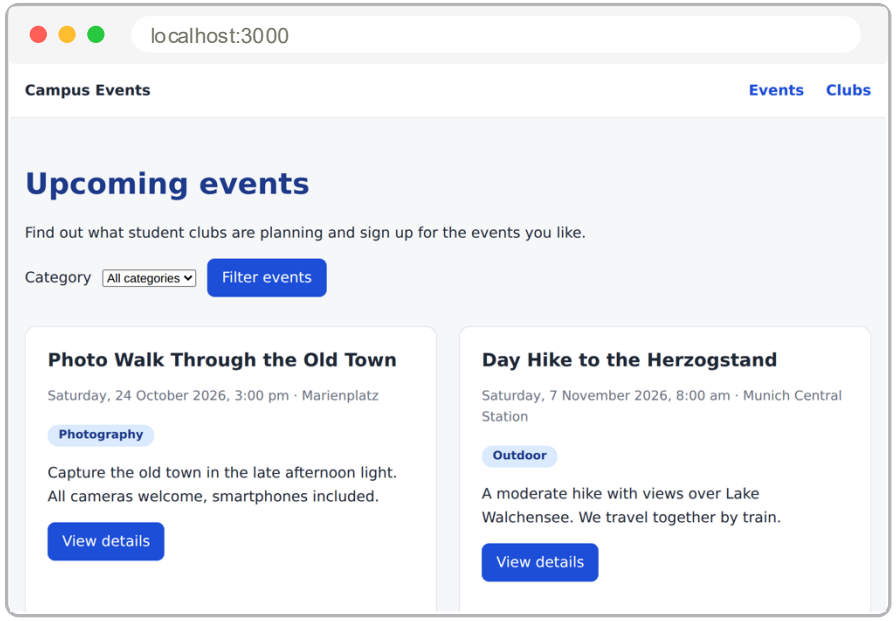
Terminal

```
$ cd campus-events
$ git pull --tags
$ npx serve public

# Zwischenstand auschecken:
$ git checkout step-7-classes-box-model

# Zurück zur neuesten Version:
$ git checkout main
```

Stand nach Vorlesung 3



Offene Fäden

Sieht schlicht aus
CSS
✓ erledigt

Der Filter tut nichts
JavaScript
Block 2

Header in jeder Datei kopiert
Komponenten
Block 3

Anmeldung endet mit 501
Backend
Block 4

Ausblick: Block 2 – JavaScript

Selektoren und Kaskade
bestimmen, welche Regel für welches Element gilt.

Box-Modell, Flexbox, Grid
sind die Werkzeuge für Abstände und Layout.

Mobile first
mit Media Queries und flexiblen Bildern.



Block 2: JavaScript

Als Erstes bekommt der Kategorie-Filter eine Funktion – mit dem DOM aus Vorlesung 2.



~/webtech $

Sie sind dran.

Im Praktikum: CSS